



NTU Academy for Professional
and Continuing Education

(SCTP) Advanced
Professional Certificate

Data Science and AI





NANYANG
TECHNOLOGICAL
UNIVERSITY
SINGAPORE

2.7 Testing and Data Orchestration

Module Overview

2.1 Introduction to Big Data and Data Engineering

2.2 Data Architecture

2.3 Data Encoding and Data Flow

2.4 Data Extraction and Web Scraping

2.5 Data Warehouse

2.6 Data Pipelines and Orchestration

2.7 Data Orchestration and Testing

2.8 Out of Core/Memory Processing

2.9 Big Data Ecosystem and Batch Processing

2.10 Event Streaming and Stream Processing

Lesson Objectives

- Orchestrate and schedule Meltano and Dbt using Dagster.
- Set up and run data quality tests using Great Expectations.
- Set up and run data quality tests using Dbt packages.

Dagster Concepts

Dagster Concept	Role with Meltano/dbt	Dagster Syntax Example
Asset	Represents dbt models as assets (tables/views).	<pre>@asset def my_dbt_model(): ...</pre>
Pipeline (Job)	Orchestrates Meltano runs and dbt model executions.	<pre>@job(resource_defs={...}) def my_pipeline(): ...</pre>
Resource	Configures access to Meltano projects, DBs, etc.	<pre>from dagster_meltano import meltano_resource</pre>
Schedule	Automates pipeline/job execution (e.g., daily runs).	<pre>@schedule(job=my_pipeline, cron_schedule="0 0 * * *")</pre>
Definitions	Registers all assets, jobs, resources, schedules.	<pre>Definitions(assets=[...], jobs=[...], resources={...})</pre>

Why do we need Orchestration

WITHOUT ORCHESTRATION

Manual pipeline execution

No dependency management

Difficult error recovery

Limited visibility

Inconsistent scheduling

WITH DAGSTER

Automated workflows

Smart dependency resolution

Automatic retries & alerts

Full pipeline observability

Reliable scheduling

Insight: Orchestration transforms ad-hoc scripts into production-ready, maintainable data pipelines

What is dbt Expectations?

dbt Expectations is a package that extends dbt's testing functionality, inspired by the Great Expectations Python library.

- Provides 50+ test macros for common data quality checks
- Allows for complex data validation scenarios
- Integrates seamlessly with dbt's testing framework
- Supports both column-level and table-level tests

Key Features

- **Column Tests:** Validate individual column values, formats, and relationships
- **Table Tests:** Verify table-level properties like row counts, uniqueness across columns
- **Multi-column Tests:** Test relationships between columns within a table
- **Schema Tests:** Ensure schema stability and expected structure

Installation

Add to your dbt packages.yml file:

```
packages:  
  - package: metaplane/dbt_expectations  
    version: 0.10.9
```

Then run:

```
dbt deps
```

Basic Usage

Add tests to your schema.yml files:

```
models:  
  - name: my_model  
    columns:  
      - name: user_id  
        tests:  
          - dbt_expectations.expect_column_values_to_not_be_null  
          - dbt_expectations.expect_column_values_to_be_unique
```


Column Value Tests

- `dbt_expectations.expect_column_values_to_be_between:`
 `min_value: 0`
 `max_value: 100`
- `dbt_expectations.expect_column_values_to_match_regex:`
 `regex: '^[A-Z]{2}$'`
- `dbt_expectations.expect_column_values_to_be_in_set:`
 `value_set: ['active', 'inactive', 'pending']`

Implementation Tips

<https://www.metaplane.dev/blog/dbt-expectations>

Table-Level Tests

- `models:`
- `name: my_model`
- `tests:`
- `dbt_expectations.expect_table_row_count_to_be_between:`
 `min_value: 1`
 `max_value: 1000`
 - `dbt_expectations.expect_table_columns_to_contain_set:`
 `column_list: ["id", "name", "created_at"]`
 - `dbt_expectations.expect_table_row_count_to_equal_other_table:`
 `compare_model: ref('other_model')`

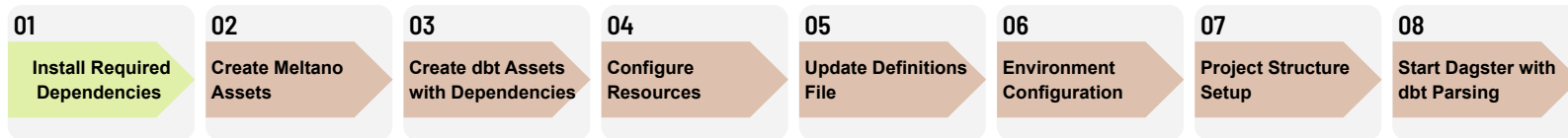
Documentation - Available Tests

https://hub.getdbt.com/metaplane/dbt_expectations/latest/

Lesson Plan

1. Part 1: Hands-on with Orchestration (Part 2)
2. Part 2: Hands-on with Data Testing with Great Expectations
3. Part 3: Testing Dbt

Dagster Configuration - Orchestrating ELT Pipelines with Meltano and DBT



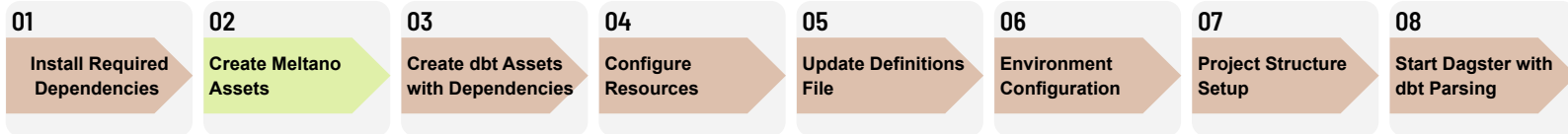
Step 1

File: Command line / `requirements.txt`

Install all necessary packages for the integration

```
pip install dagster dagster-webserver  
dagster-meltano dagster-dbt dbt-bigquery
```

Dagster Configuration - Orchestrating ELT Pipelines with Meltano and DBT



Step 2

File: `assets/meltano_assets.py`

Purpose: Define EL operations as Dagster assets using Meltano

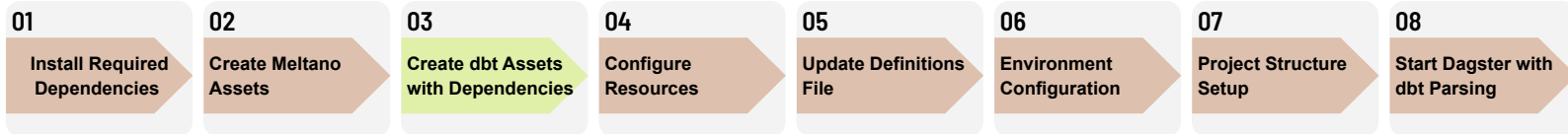
```
from dagster import asset, AssetExecutionContext
from dagster_meltano import MeltanoResource
from pathlib import Path

MELTANO_PROJECT_DIR = Path(__file__).parent.parent / "meltano"

@asset(group_name="extract_load")
def raw_customer_data(context: AssetExecutionContext, meltano:
MeltanoResource):
    """Extract customer data to BigQuery using Meltano"""
    result = meltano.run(["el", "tap-csv", "target-bigquery"])
    context.log.info(f"Customer data EL completed: {result}")
    return result

@asset(group_name="extract_load")
def raw_sales_data(context: AssetExecutionContext, meltano:
MeltanoResource):
    """Extract sales data to BigQuery using Meltano"""
    result = meltano.run(["el", "tap-salesforce",
"target-bigquery"])
    context.log.info(f"Sales data EL completed: {result}")
    return result
```

Dagster Configuration - Orchestrating ELT Pipelines with Meltano and DBT



Step 3

File: `assets/dbt_assets.py`

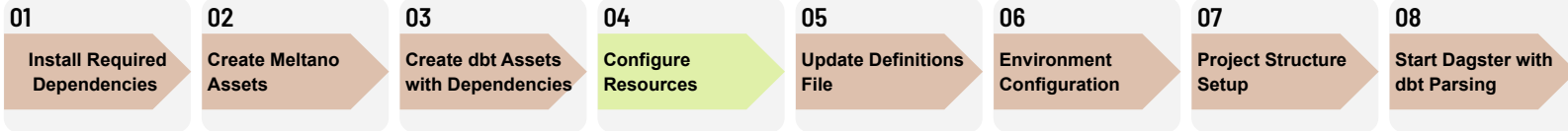
Purpose: Define dbt transformations as assets that depend on Meltano EL completion

```
from dagster import AssetExecutionContext
from dagster_dbt import DbtCliResource, dbt_assets,
DbtProject
from pathlib import Path
from .meltano_assets import raw_customer_data,
raw_sales_data

DBT_PROJECT_DIR = Path(__file__).parent.parent / "dbt"
dbt_project = DbtProject(project_dir=DBT_PROJECT_DIR)

@dbt_assets(
    manifest=dbt_project.manifest_path,
    project=dbt_project,
    name="all_dbt_models",
    group_name="transform",
    deps=[raw_customer_data, raw_sales_data]  # EL
    dependencies
)
def all_dbt_transformations(context:
AssetExecutionContext, dbt: DbtCliResource):
    """Run all dbt transformations after EL operations
    complete"""
    yield from dbt.cli(["build"],
context=context).stream()
```

Dagster Configuration - Orchestrating ELT Pipelines with Meltano and DBT



Step 4

File: `resources.py`

Purpose: Configure Meltano and dbt resources for Dagster

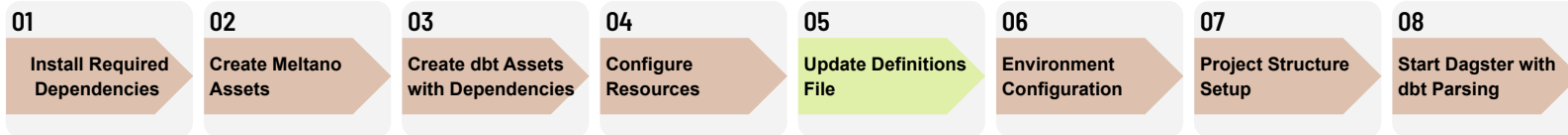
```
from dagster_meltano import MeltanoResource
from dagster_dbt import DbtCliResource
from pathlib import Path

MELTANO_PROJECT_DIR = Path(__file__).parent /
    "meltano"
DBT_PROJECT_DIR = Path(__file__).parent / "dbt"

meltano_resource = MeltanoResource(
    project_dir=str(MELTANO_PROJECT_DIR)
)

dbt_resource = DbtCliResource(
    project_dir=str(DBT_PROJECT_DIR),
    target="prod"
)
```

Dagster Configuration - Orchestrating ELT Pipelines with Meltano and DBT



Step 5

File: `definitions.py`

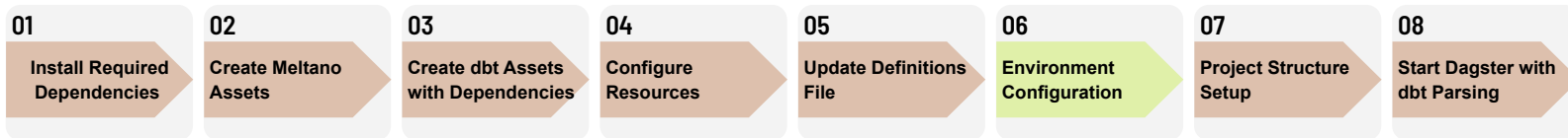
Purpose: Combine all assets and resources into Dagster definitions

```
from dagster import Definitions, load_assets_from_modules
from . import assets
from .resources import meltano_resource, dbt_resource

all_assets = load_assets_from_modules([assets])

defs = Definitions(
    assets=all_assets,
    resources={
        "meltano": meltano_resource,
        "dbt": dbt_resource,
    }
)
```

Dagster Configuration - Orchestrating ELT Pipelines with Meltano and DBT



Step 6

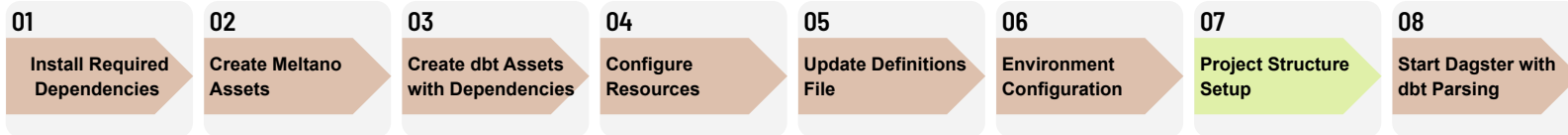
File: `.env` or environment variables

Purpose: Configure environment for dbt parsing and BigQuery

```
# Enable dbt project parsing on load
DAGSTER_DBT_PARSE_PROJECT_ON_LOAD=1

# Optional BigQuery configuration
GOOGLE_APPLICATION_CREDENTIALS=/path/to/service-account.json
DBT_PROFILES_DIR=../dbt/profiles
```


Dagster Configuration - Orchestrating ELT Pipelines with Meltano and DBT



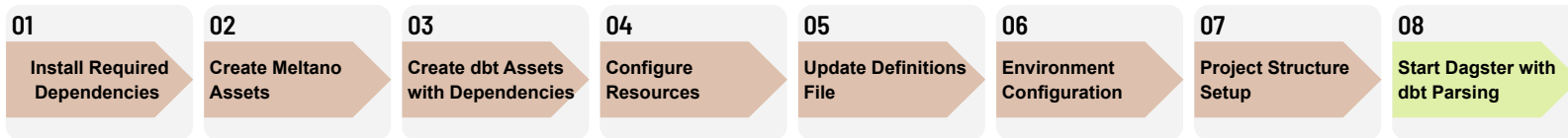
Step 7

File: Project directory structure

Purpose: Ensure proper directory organization

```
your_project/
├── assets/
│   ├── __init__.py
│   ├── meltano_assets.py
│   └── dbt_assets.py
├── meltano/
│   └── (meltano project files)
├── dbt/
│   └── (dbt project files)
├── resources.py
├── definitions.py
└── .env
```

Dagster Configuration - Orchestrating ELT Pipelines with Meltano and DBT



Step 8

Action: Launch Dagster development server

Command: Terminal execution

```
DAGSTER_DBT_PARSE_PROJECT_ON_LOAD=1 dagster dev
```

End of Lesson - Exit Ticket

Survey Link

<https://www.menti.com>

